

Parte 2: Aprendiendo el freeRTOS

- [Gestion de tareas](#)
- [Colas o Queues \(Productor-Consumidor\)](#)
- [Semaforos y MUTEX](#)
- [Notificaciones directas \(Task notifications\)](#)
- [Grupos de eventos](#)

Desarrollo de temas

Creacion de tareas (Task create)

Librerias: `#include "freertos/FreeRTOS.h"` y `#include "freertos/task.h"`

Funcion para crear una tarea

```
static inline BaseType_t xTaskCreate(TaskFunction_t pxTaskCode, const char *const
pcName, const configSTACK_DEPTH_TYPE usStackDepth, void *const pvParameters, UBaseType_t
uxPriority, TaskHandle_t *const pxCreatedTask)
```

Esta funcion utiliza memoria dinamica para crearse. Su funcion contraparte utiliza no usa memoria dinamica, `xTaskCreateStatic`

---> Parametros

- `pxTaskCode`: Puntero a la funcion a ejecutar. Recordar que esta funcion no debe terminar nunca y tampoco debe retornar. Su prototipo es `void func(void* args)`
- `pcName`: Puntero al string con el nombre. Esta debe ser menor a 16 caracteres o 16 bytes
- `usStackDepth`: Tamaño del stack, un tamaño de 2048 para tareas simples esta bien. Usar 4096 si la funcion usa `printf` o logs
- `pvParameters`: Parametros que se le pasan a la funcion. En la practica, para pasarle muchos parametros a una funcion, es mejor encapsularla en una estructura. Para que esa estructura no se pierda, se debe crearla con memoria dinamica usando `malloc()`
- `uxPriority`: Nivel de prioridad siendo 0 la prioridad mas baja y 24 la mas alta
- `pxCreatedTask`: Puntero al handle de la tarea

---> Retorna

`pdPASS` si la tarea se creo correctamente o un error definido en el archivo `projdefs.h`

Funcion para crear una tarea en un nucleo especifico

```
BaseType_t xTaskCreatePinnedToCore(TaskFunction_t pxTaskCode, const char *const pcName,
const uint32_t ulStackDepth, void *const pvParameters, UBaseType_t uxPriority,
TaskHandle_t *const pxCreatedTask, const BaseType_t xCoreID)
```

---> Parametros

- Los mismos que la función `xTaskCreate()`
- `xCoreID`: El núcleo en el cual la tarea es fijada. Comúnmente para Wifi/BT usar el pin 0. Si quiero que el sistema decida el pin, usar la macro `tskNO_AFFINITY`

Función para eliminar una tarea

```
void vTaskDelete(TaskHandle_t xTaskToDelete)
```

---> Parámetros

- `xTaskToDelete`: Manejador de la tarea a eliminar. Si la función se aplica sobre la tarea misma, usar `NULL`

Función para aplicar un delay a la tarea

```
void vTaskDelay(const TickType_t xTicksToDelay)
```

---> Parámetros

- `xTicksToDelay`: Tiempo, en ticks, en el que la tarea se bloquea. La macro `pdMS_TO_TICKS(ms)` convierte milisegundos a ticks del sistema

Colas o queue

Librería: `#include "freertos/queue.h"`

Macro para crear la cola

```
xQueueCreate(uxQueueLength, uxItemSize)
```

---> Parámetros

- `uxQueueLength`: Tamaño de la cola o cantidad de elementos
- `uxItemSize`: Tamaño, en bytes, de cada elemento o ítem

---> Retorna

- El handle o manejador de la cola
- Si no se pudo crear, retorna 0

Macro para añadir un elemento al fondo

```
xQueueSend(xQueue, pvItemToQueue, xTicksToWait)
```

Su macro contrario es `xQueueSendToFront` que añade el ítem al inicio

---> Parámetros

- `xQueue`: Manejador de la cola
- `pvItemToQueue`: Puntero al ítem a guardar
- `xTicksToWait`: Tiempo, en ticks, que debe esperar la tarea para añadir un elemento en caso de que la cola esté llena

---> Retorna

- pdTRUE: En caso de que el item se pudo añadir
- errQUEUE_FULL: Otro error

Funcion para obtener un item de la cola

```
BaseType_t xQueueReceive(QueueHandle_t xQueue, void *const pvBuffer, TickType_t xTicksToWait)
```

---> Parametros

- xQueue: Manejador de la cola
- pvBuffer: Puntero al buffer donde se guarda el elemento
- xTicksToWait: Tiempo, en ticks, que debe esperar la tarea para retirar un elemento en caso de que la cola este vacia

---> Retorna

- pdTRUE: Si el item se pudo retirar de la cola
- pdFALSE: Si hubo un error

Uso en Interrupciones (ISR)

Las funciones xQueueSendFromISR y xQueueReceiveFromISR tienen un parámetro extra vital: `BaseType_t *pxHigherPriorityTaskWoken`

¿Qué es?: Un puntero a una variable booleana.

¿Para qué sirve?: La función pondrá esta variable en pdTRUE si la operación despertó a una tarea de mayor prioridad que la que se interrumpió.

¿Qué debo hacer?: Si la variable termina en pdTRUE, debes llamar a `portYIELD_FROM_ISR()` al final de la interrupción para forzar el cambio de contexto inmediato.

Semaforos y MUTEX

Libreria: `#include "freertos/semphr.h"`

Macro para crear el semaforo binario

```
xSemaphoreCreateBinary()
```

---> Retorna

- Manejador del semaforo binario, debe ser del tipo `SemaphoreHandle_t`, este inicia vacio por lo que le tenemos que dar un primer valor para que arranque disponible

Macro para crear el MUTEX

xSemaphoreCreateMutex()

---> **Retorna**

- Manejador del mutex, debe ser del tipo `SemaphoreHandle_t`, este inicia disponible

Macro para bloquear el semaforo

xSemaphoreTake(xSemaphore, xBlockTime)

El termino bloquear o "Lock" en ingles viene de que en C, el mutex se bloqua y se desbloquea. Capaz con este termino se entiende mejor el uso

---> **Parametros**

- xSemaphore: Manejador del semaforo
- xBlockTime: Tiempo, en ticks, que debe esperar hasta que el semaforo este disponible

---> **Retorna**

- pdTRUE: Si el semaforo fue obtenido
- pdFALSE: Si el tiempo expiro sin que el semaforo este disponible

Macro para liberar el semaforo

xSemaphoreGive(xSemaphore)

---> **Parametros**

- xSemaphore: Manejador del semaforo

---> **Retorna**

- pdTRUE: Si el semaforo fue liberado
- pdFALSE: Si ocurrio un error

Caracteristica	Semaforo binario	Mutex
Uso Principal	Sincronización: Una tarea (o ISR) avisa a otra que algo pasó.	Protección (Exclusión Mutua): Proteger una variable o bus para que solo uno la use a la vez.
Dueño (Ownership)	NO. Cualquiera puede dar o tomar.	SÍ. Solo quien lo toma puede liberarlo.
Herencia de Prioridad	NO. Sufre de "Inversión de Prioridad".	SÍ. Evita bloqueos si tareas de distinta prioridad compiten.
Uso en ISR	Permitido (<code>GiveFromISR</code>).	PROHIBIDO. No se puede usar Mutex dentro de una interrupción.

Notificaciones directas (Task notifications)

Libreria: Misma que semaforo

Macro para dar la notificacion

```
xTaskNotifyGive(xTaskToNotify)
```

---> **Parametros**

- xTaskToNotify: Manejador de la tarea a notificar. Es el mismo del que retorna `xTaskCreate()`

---> **Retorna**

- Siempre retorna `pdPASS`

Razón técnica: Esta función simplemente incrementa un contador dentro de la tarea destino. A diferencia de una cola, no puede "llenarse" ni fallar por falta de memoria, así que siempre tiene éxito.

Macro para recibir una notificacion

```
ulTaskNotifyTake(xClearCountOnExit, xTicksToWait)
```

---> **Parametros**

- xClearCountOnExit: Si es `pdFALSE` entonces el valor de la notificacion es decrementado hasta que la funcion salga. Caso contrario, la notificacion de la tarea se setea a cero cuando la funcion salga
- xTicksToWait: Tiempo, en ticks, que la tarea debe esperar bloqueado para que el valor de la notificacion sea mayor a cero

---> **Retorna**

- El recuento de notificaciones de la tarea antes de que se borre a cero o se reduzca

Necesidad	¿Usar Notificación?	¿Por qué?
Sincronizar ISR con Tarea	☒ SÍ	Es la opción más rápida y ligera. Reemplaza al Semáforo Binario
Pasar datos (char, struct)	☒ NO	Usa una Cola. La notificación solo guarda 32 bits
Sincronizar varias tareas	☒ NO	Usa Event Groups o Semáforos. La notificación es privada de una tarea
Contar eventos	☒ SÍ	Funciona perfecto como semáforo contador sin gastar RAM

Grupos de eventos

```
Libreria: #include "freertos/event_groups.h"
```

Manejador del grupo de eventos

EventGroupHandle_t

Funcion para crear un grupo de eventos

EventGroupHandle_t xEventGroupCreate(void)

---> **Retorna**

- El manejador de eventos
 - NULL si hubo memoria insuficiente en el HEAP
-

Ejemplo de como definir los bits

El ESP32 es de 32 bits, pero FreeRTOS reserva los 8 bits superiores. Solo puedes usar 24 bits (del 0 al 23).

```
#define WIFI_CONNECTED_BIT    (1 << 0) // 001
#define MQTT_CONNECTED_BIT    (1 << 1) // 010
#define BUTTON_PRESSED_BIT    (1 << 2) // 100
```

Funcion para setear bits

EventBits_t xEventGroupSetBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToSet)

---> **Parametros**

- EventGroupHandle_t: El manejador del grupo de eventos en la que los bits se van a setear
- uxBitsToSet: Mascara de bits indicando los bits a setear

---> **Retorna**

- El valor del grupo de eventos al momento de retornar la llamada

Funcion para esperar el grupo

EventBits_t xEventGroupWaitBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToWaitFor, const BaseType_t xClearOnExit, const BaseType_t xWaitForAllBits, TickType_t xTicksToWait)

---> **Parametros**

- xEventGroup: Manejador del grupo de eventos
- uxBitsToWaitFor: La mascara de bits con los que estoy esperando
- xClearOnExit: **pdTRUE** para limpiar (borrar) los bits al salir (útil para sincronizar). **pdFALSE** para dejarlos intactos (útil para leer estados)
- xWaitForAllBits: **pdTRUE** hace que espere **todos** los bits definidos esten en 1. **pdFALSE** espera a que **cualquiera** que los bits definidos este en 1

- xTicksToWait: Tiempo maximo de espera

---> Retorna

- El valor de los bits del grupo en el momento en que se desbloqueó la tarea (o expiró el tiempo). Sirve para saber qué bit exacto causó el despertar

Funcion para limpiar los bits

```
EventBits_t xEventGroupClearBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear)
```

---> Parametros

- xEventGroup: Manejador del grupo de eventos
- uxBitsToClear: Mascara de bits a colocar en 0

---> Retorna

- El valor del grupo de eventos antes de que se borrarán los bits especificados

Lógica de Funcionamiento: Productores y Consumidores

Imagina que tienes 3 Tareas (Productores) y 1 Tarea Principal (Consumidor) que no puede arrancar hasta que todo esté listo.

Quien	Accion	Codigo	Bit en el grupo
Tarea Wifi	Conectado	<code>xEventGroupSetBits(grupo, BIT_0)</code>	001
Tarea server	Encontrado	<code>xEventGroupSetBits(grupo, BIT_1)</code>	010
Tarea sensor	Calibrado	<code>xEventGroupSetBits(grupo, BIT_2)</code>	100
Tarea principal	Duerme esperando a todos	<code>xEventGroupWaitBits(grupo, BIT_0 - BIT_1 - BIT_2)</code>	BIT_1